Lucrul cu stiva și apeluri de funcții

Stiva programului

- procesorul foloseşte o parte din memoria RAM pentru a o accesa printr-o disciplină de tip LIFO
  - gestiune - adresa vârfului stivei
  - memorată în registrul ESP
- structurile de tip stivă sunt necesare în multe situaţii

Introducere în stivă

- sintaxă

`push` *`operand`*

- operandul poate fi doar pe 2 sau 4 octeți
  - se recomandă lucrul numai cu operanzi pe 4 octeţi
- valoarea ESP este scăzută cu dimensiunea operandului (2 sau 4)
  - apoi operandul este copiat în vârful stivei

Exemple

```
push eax
push dx
push dword ptr [200]
push word ptr [esi*2+ecx]
push dword ptr 5
push word ptr 14
```

Extragere din stivă

- sintaxă

`pop` *`destinatie`*

- destinația poate fi tot pe 2 sau 4 octeţi
- valoarea din vârful stivei (ESP) se depune la destinație
- apoi, la valoarea ESP se adună numărul de octeţi ai operandului

Exemple

```
pop eax
pop cx
pop dword ptr [ebx+edi]
pop word ptr [edx+100]
```

- destinația nu poate fi o constantă

Utilizare practică

- rolul stivei programului - stocarea de informaţii cu caracter temporar
  - salvarea pentru un timp a valorii unui registru, pentru a-l folosi în alt scop
    - apoi valoarea salvată este restaurată în registru
  - variabile locale
    - create la momentul apelului funcţiei și distruse la terminarea execuției funcţiei

Atenție

- instrucţiunile de introducere în stivă trebuie riguros compensate de cele de scoatere din stivă
  - ca număr de instrucţiuni
  - ca dimensiune a operanzilor
- orice eroare afectează mai multe date decât pare la prima vedere

Exemplu (1)

```
push eax
push edx
mov eax,[esi]
mov edx,5
mul edx
mov [esi],eax
pop eax
```

Exemplu (2)

- o instrucţiune `pop` a fost uitată - efecte
  - registrul `eax` primeşte altă valoare decât avea iniţial
  - valoarea iniţială a registrului `edx` nu este restaurată
- similar dacă se execută prea multe instrucţiuni `pop`

Apeluri de funcții

- la prima vedere - ca o instrucţiune de salt
  - se întrerupe execuţia liniară a programului şi se sare la altă adresă
- diferență - la terminarea funcţiei se revine la adresa de unde s-a făcut apelul
  - deci aceasta a fost memorată
  - informaţie temporară - tot pe stivă

Instrucțiunea de apel

- sintaxă

`call` *`adresa`*

  – în Visual C++, adresa este indicată folosind chiar numele funcţiei apelate

- efectul instrucţiunii

  – se introduce în stivă adresa instrucţiunii următoare (adresa de revenire)

  – se face salt la adresa indicată în instrucțiune

Revenirea din funcție

- sintaxă

```
ret
```

- efectul instrucţiunii
  - preia adresa de revenire din vârful stivei
  - face saltul la adresa respectivă

Erori în lucrul cu stiva

- o instrucţiune `pop` omisă (sau una în plus)
  - la execuţia unei instrucţiuni `ret` se va prelua din stivă altă valoare decât adresa corectă de revenire
  - efect - blocarea programului sau terminarea sa forţată

Parametrii funcțiilor (1)

- sunt tot variabile locale - se găsesc pe stivă
- apelantul are responsabilitatea
  - de a-i pune pe stivă la apel
    - folosind instrucțiunea `push`
  - de a-i scoate de pe stivă la revenirea din funcţia apelată
    - nu se folosește instrucțiunea `pop`, ci se adună la ESP numărul total de octeţi ocupat de parametri

Parametrii funcțiilor (2)

- în C/C++ parametrii trebuie puşi în stivă în ordine inversă celei în care se găsesc în lista de parametri
  - motiv - funcții cu număr variabil de parametri
- de ce trebuie scoşi din stivă la revenire?
  - la finalul fiecărei unități de program trebuie să lăsăm stiva așa cum am găsit-o la intrare
  - altfel apar erori de tipul celor descrise anterior

Exemplu (1)

- cod C

```
void dif(int a,int b)
{
  int c;
  c=a-b;
  cout<<c<<endl;
}
```

Exemplu (2)

- apelul `dif(9,5)` se traduce prin secvenţa de mai jos:

```
push dword ptr 5
push dword ptr 9
call dif
add esp,8
```

Accesul la parametri

- Visual C++ - prin numele lor
  - funcții cu număr variabil de parametri?
- în general - prin adresele lor
  - primul parametru din antetul funcţiei se găseşte întotdeauna la adresa EBP+8
  - următorii parametri, mergând spre dreapta în lista parametrilor, sunt la adrese corespunzător mai mari: EBP+12, EBP+16 etc.

Funcții - returnarea valorilor

- convenție stabilită de compilator
- Visual C++ - rezultatul se depune într-un registru - depinde de dimensiunea sa
  - tipuri de date de dimensiune 1 octet: `al`
  - tipuri de date de dimensiune 2 octeţi: `ax`
  - tipuri de date de dimensiune 4 octeţi: `eax`
  - tipuri de date de dimensiune 8 octeţi: `edx` şi `eax`

Păstrarea valorilor regiştrilor

- programul pe care îl scriem este C/C++
- nu ştim cum foloseşte compilatorul regiştrii
- pentru orice funcţie pe care o scriem în limbaj de asamblare
  - la începutul execuţiei funcţiei trebuie salvați în stivă regiștrii pe care îi folosim în funcție
  - la terminare, valorile lor trebuie restaurate
  - excepție - `eax` şi `edx` (returnare valori)